Circuit Design
Схемотехника

Melis Sydykbekovich Osmonov,
Candidate of Technical Sciences,
KRSU,IVT

© Osmonov M.S. KRSU

Введение

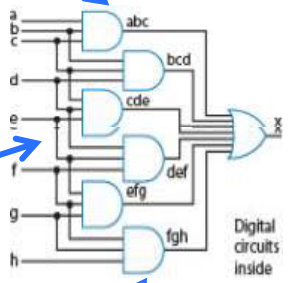
© Osmonov M.S.
KRSU

Outline

- Introduction

Why Study Digital Design?











- Look “under the hood” of computers
  - Solid understanding --> confidence, insight, even better programmer when aware of hardware resource issues
- Electronic devices becoming digital
  - Enabled by shrinking and more capable chips
  - Enables:
    - Better devices: Better sound recorders, cameras, cars, cell phones, medical devices,...
    - New devices: Video games, PDAs, ...
  - Known as “embedded systems”
    - Thousands of new devices every year
    - Designers needed: Potential career direction

Portable music players	Satellites	Cell phones	DVD players	Cameras	Video recorders	TVs	Musical instruments
1995	1997	1999	2001	2003	2005	2007	???

Digital Design: Motivation

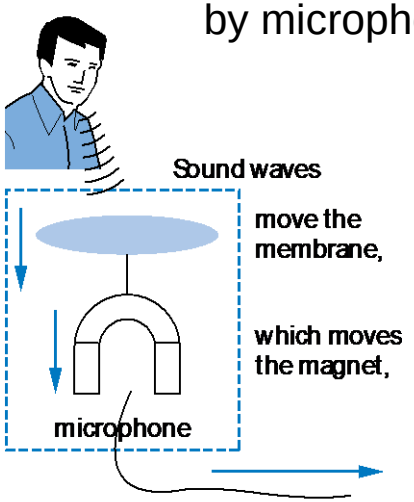
- Implementation basis for modern computing devices
  - Constructing large systems from small components
  - Another view of a computer: ***controller + datapath***
- Inherent parallelism in hardware
- Counterpoint to software design
  - Furthering our understanding of computation

Digital Design

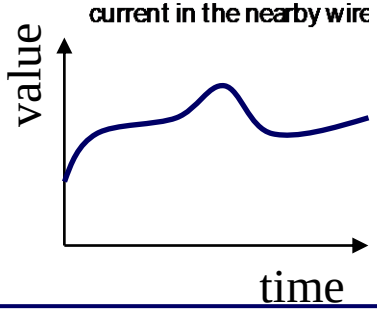
- What is digital ?
  - Digital camera, Digital TV, Digital Watch, Digital
- Radio, Digital City (e-city), Digital Photo Frame …etc.
  - Which gives the things in countable form
  - Scene (analog) to Image (digital)
- Why digital ?
  - Countable form, makes easy to manage
  - Easy management makes more useful and versatile
- What digit ?
  - How to count: Decimal digit: 0 to 9

What Does “Digital” Mean?

- Analog signal
  - Infinite possible values
    - Ex: voltage on a wire created by microphone

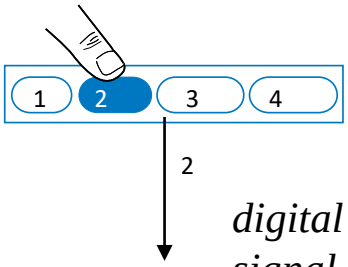


analog signal

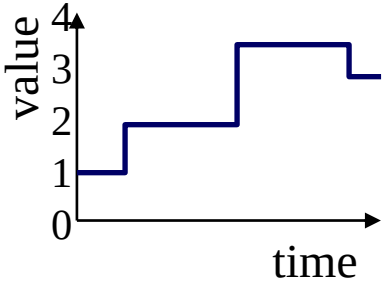


Possible values:
1.00, 1.01, 2.0000009, ... infinite possibilities

- Digital signal
  - Finite possible values
    - Ex: button pressed on a keypad



digital signal



Possible values:
0, 1, 2, 3, or 4.
That’s it.

© Osmonov M.S.
KRSU

What Digit? => Number System

- Famous Number System: Dec, Rom, Bin
- Decimal System: 0 -9
  - May evolves: because human have 10 finger
- Roman System
  - May evolves to make easy to look and feel
  - Pre/Post Concept: (IV, V & VI) is (5-1, 5 & 5+1)
- Binary System, Others (Oct, Hex)
  - One can cut an apple in to two

© Osmonov M.S.
KRSU

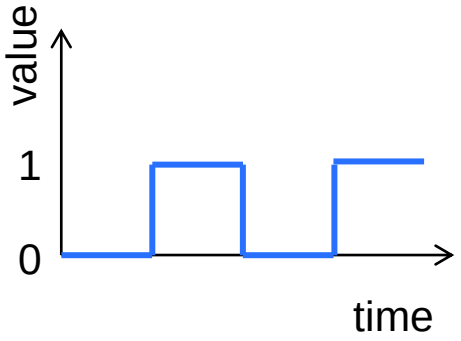
Design & Logic Design

- What is design?
  - Given problem spec, solve it with available components
  - While meeting quantitative (size, cost, power) and qualitative (beauty, elegance)
- What is logic design?
  - Choose digital logic components to perform specified control, data manipulation, or communication function and their interconnection
  - Which logic components to choose?
    - Many implementation technologies (fixed-function components, *programmable devices*, individual transistors on a chip, etc.)
  - Design optimized/transformed to meet design constraints

© Osmonov M.S.
KRSU

Digital Signals with Only Two Values: Binary

- ***Binary*** digital signal -- only *two* possible values
  - Typically represented as **0** and **1**
  - One *B*inary dig*it* is a ***bit***
  - We'll only consider *binary* digital signals
  - Binary is popular because
    - Transistors, the basic digital electric component, operate using *two* voltages
    - Storing/transmitting one of *two* values is easier than three or more (e.g., loud beep or quiet beep, reflection or no reflection)



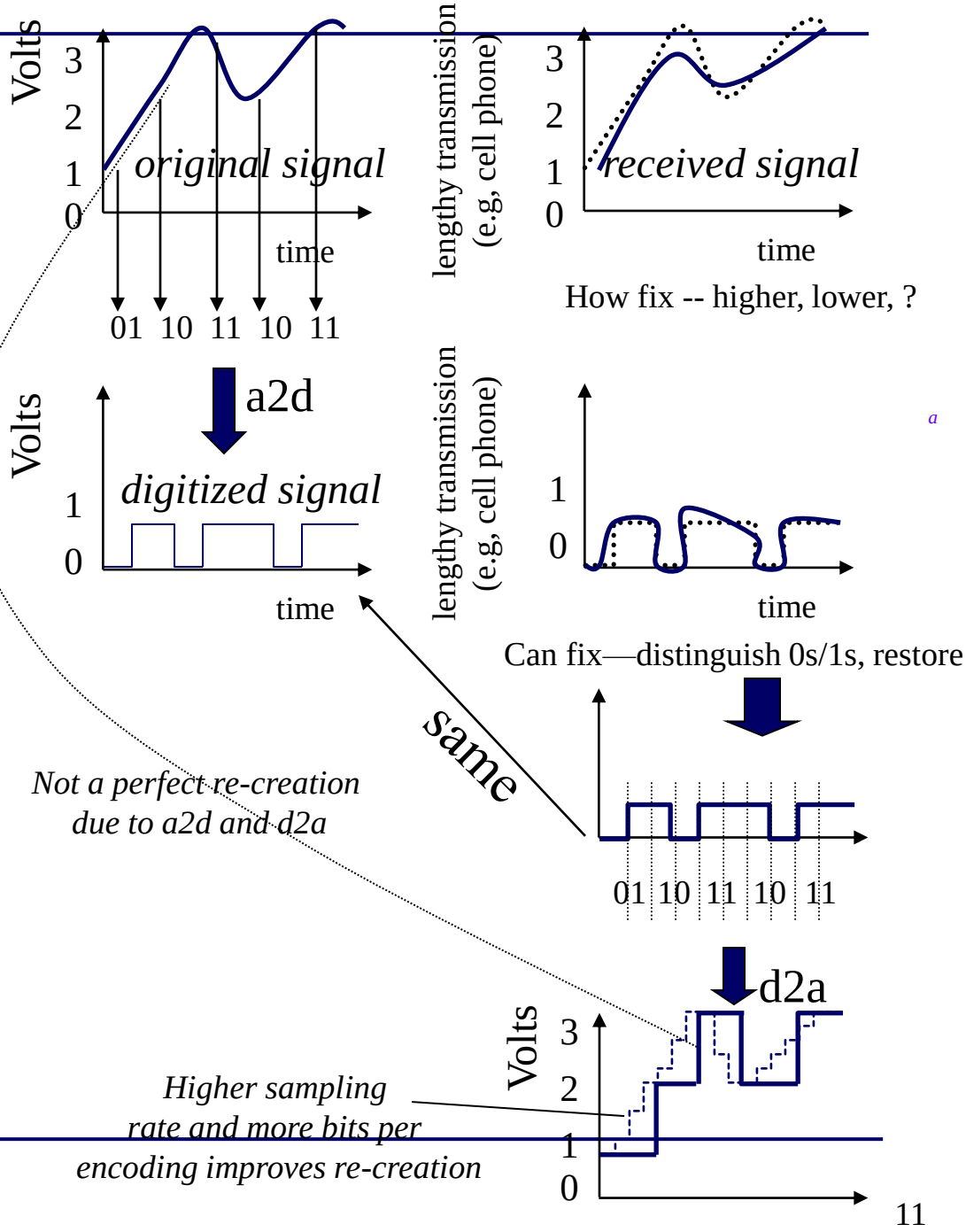
© Osmonov M.S.
KRSU

Example of Digitization Benefit

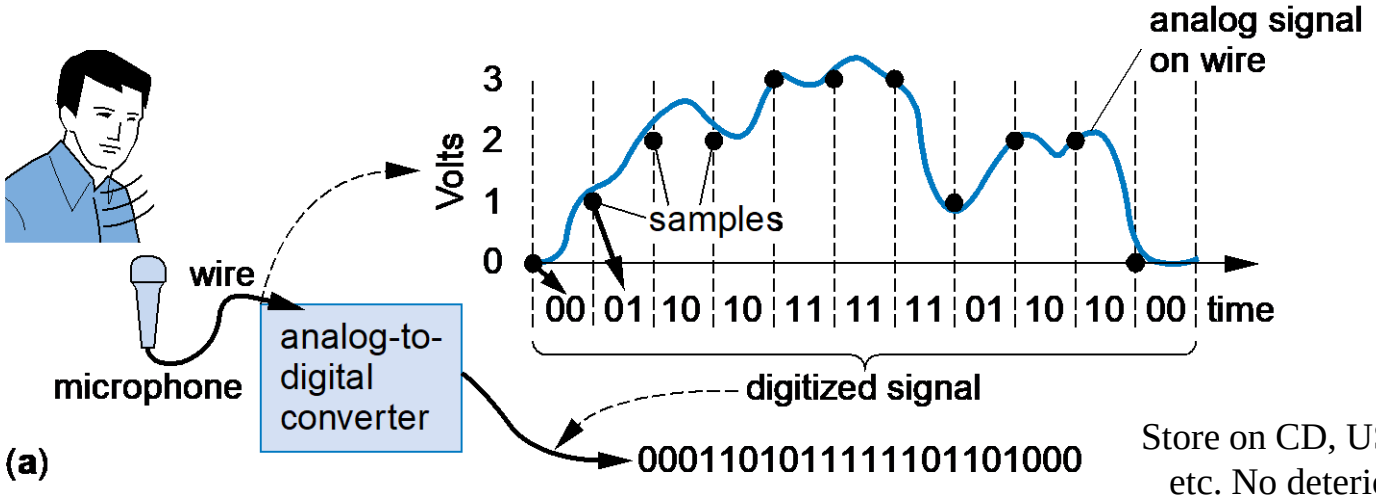
- Analog signal (e.g., audio, video) may lose quality
  - Voltage levels not saved/copied/transmitted perfectly
- Digitized version enables near-perfect save/cpy/tran.
  - "Sample" voltage at particular rate, save sample using bit encoding
  - Voltage levels still not kept perfectly
  - But we can distinguish 0s from 1s

Let bit encoding be:

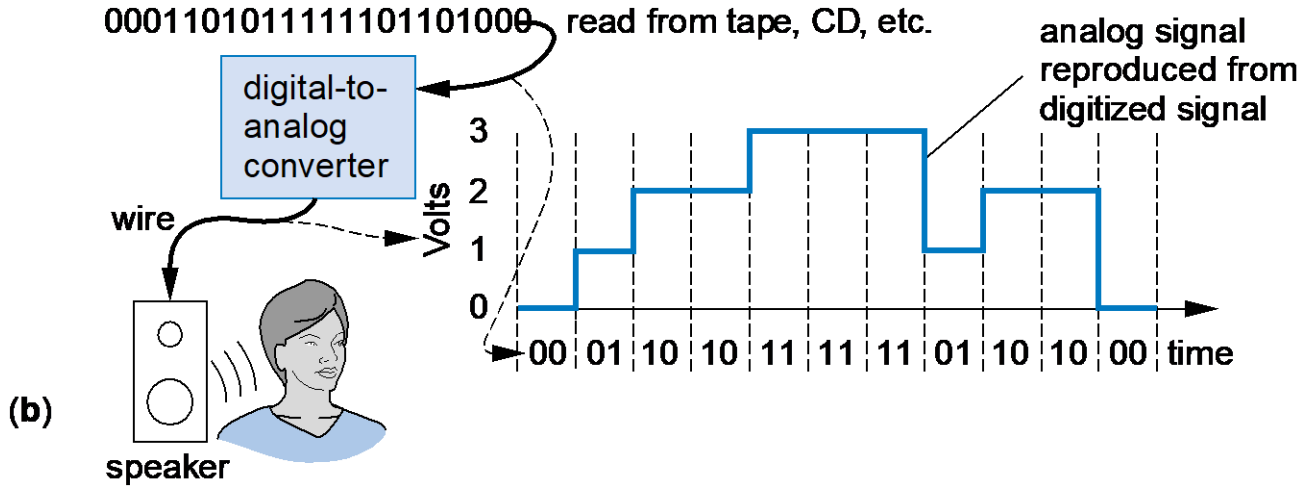
1 V: "01"
2 V: "10"
3 V: "11"



Digitization Benefit: Can Store on Digital Media



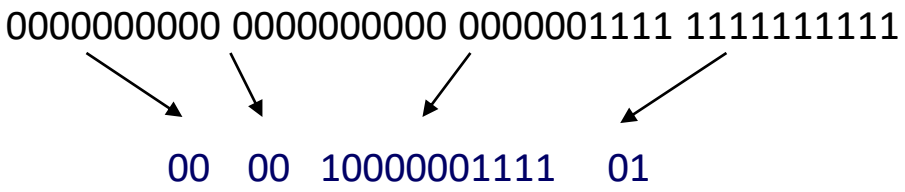
Store on CD, USB drive, etc. No deterioration.



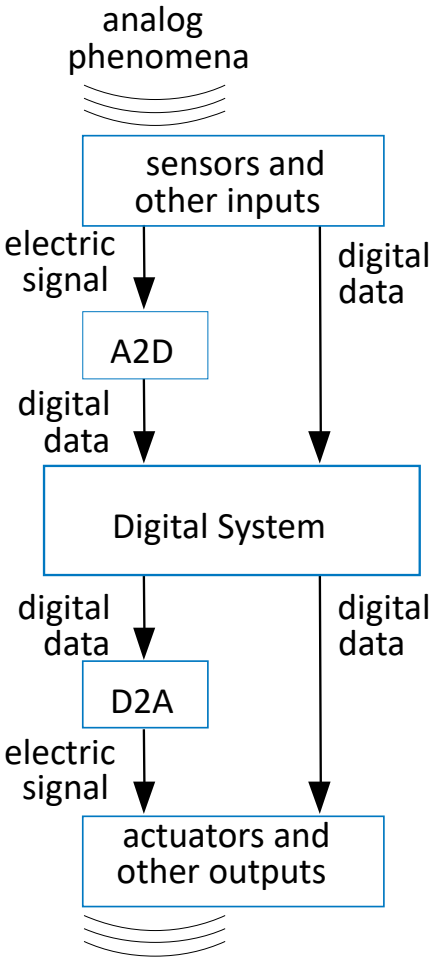
Digitized Audio: Compression Benefit

- Digitized audio can be compressed
  - e.g., MP3s
  - A CD can hold about 20 songs uncompressed, but about 200 compressed
- Compression also done on digitized pictures (jpeg), movies (mpeg), and more
- Digitization has many other benefits too

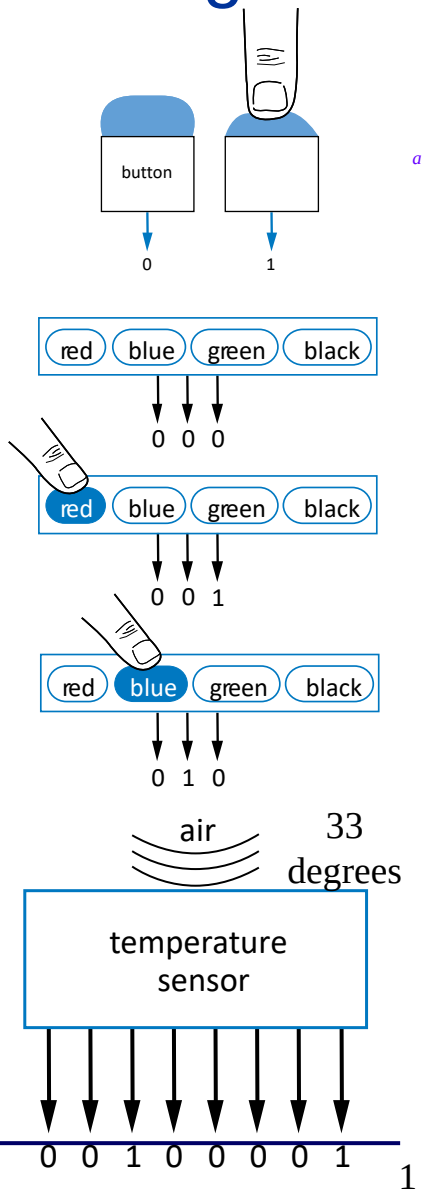
Example compression scheme:
00 means 0000000000
01 means 1111111111
1X means X



How Do We Encode Data as Binary for Our Digital System?



- Some inputs inherently binary
  - Button: not pressed (0), pressed (1)
- Some inputs inherently digital
  - Just need encoding in binary
  - e.g., multi-button input: encode red=001, blue=010, ...
- Some inputs analog
  - Need analog-to-digital conversion
  - As done in earlier slide -- sample and encode with bits



How to Encode Text: ASCII, Unicode

- ASCII: 7- (or 8-) bit encoding of each letter, number, or symbol
- Unicode: Increasingly popular 16-bit encoding
  - Encodes characters from various world languages

Sample ASCII encodings

Encoding	Symbol
010 0000	<space>
010 0001	!
010 0010	"
010 0011	#
010 0100	$
010 0101	%
010 0110	&
010 0111	'
010 1000	(
010 1001	)
010 1010	*
010 1011	+
010 1100	,
010 1101	-
010 1110	.
010 1111	/

Encoding	Symbol
100 0001	A
100 0010	B
100 0011	C
100 0100	D
100 0101	E
100 0110	F
100 0111	G
100 1000	H
100 1001	I
100 1010	J
100 1011	K
100 1100	L
100 1101	M

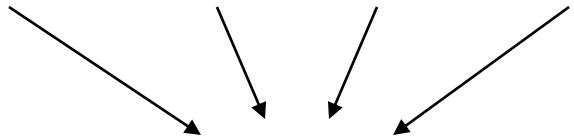
Encoding	Symbol
100 1110	N
100 1111	O
101 0000	P
101 0001	Q
101 0010	R
101 0011	S
101 0100	T
101 0101	U
101 0110	V
101 0111	W
101 1000	X
101 1001	Y
101 1010	Z

Encoding	Symbol
110 0001	a
110 0010	b
...	
111 1001	y
111 1010	z
011 0000	0
011 0001	1
011 0010	2
011 0011	3
011 0100	4
011 0101	5
011 0110	6
011 0111	7
011 1000	8
011 1001	9

Question:

What does this ASCII bit sequence represent?

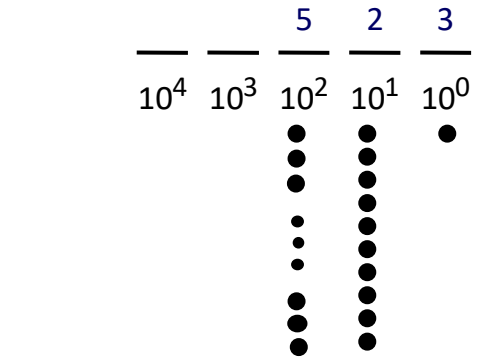
1010010 1000101 1010011 1010100

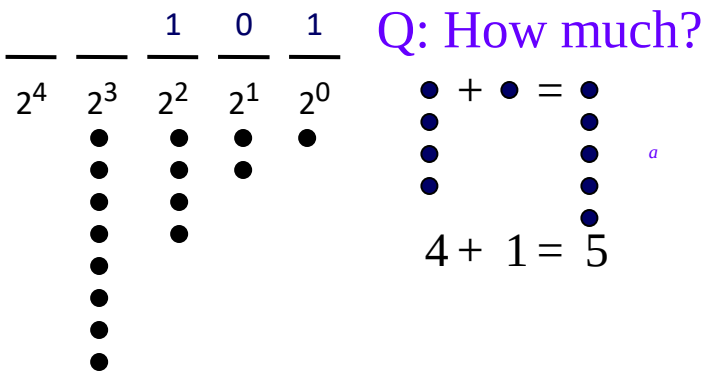


R E S T

How to Encode Numbers: Binary Numbers

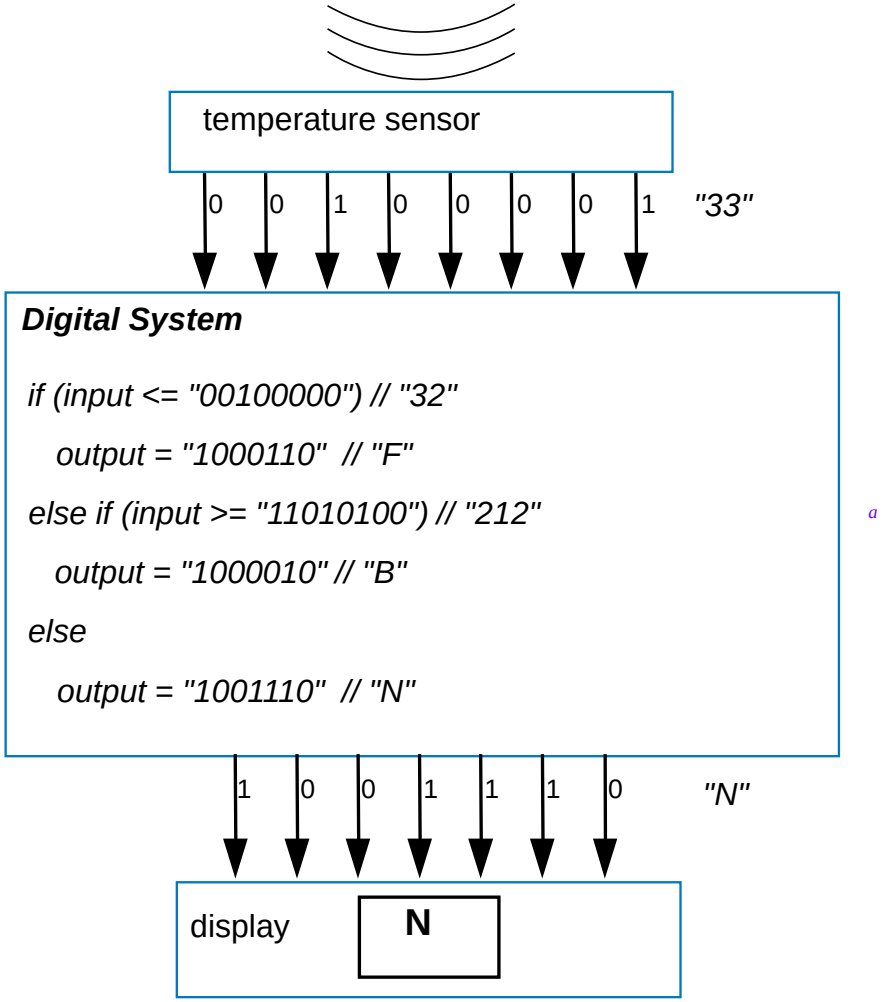
- Each position represents a quantity; symbol in position means how many of that quantity
  - Base ten (***decimal***)
    - Ten symbols: 0, 1, 2, ..., 8, and 9
    - More than 9 -- next position
      - So each position power of 10
    - Nothing special about base 10 -- used because we have 10 fingers
  - Base two (***binary***)
    - Two symbols: 0 and 1
    - More than 1 -- next position
      - So each position power of 2





Using Digital Data in a Digital System

- A temperature sensor outputs temperature in binary
- The system reads the temperature, outputs ASCII code:
  - “F” for freezing (0-32)
  - “B” for boiling (212 or more)
  - “N” for normal
- A display converts its ASCII input to the corresponding letter



Advance Digital Logic Design

- Complicated Circuit (Processor, Memory)
- IC, MSI, LSI, VLSI, ULSI…
- CAD Tool to accelerate
  - Design, Model, Simulate, Validate, Test
  - Placing, Routing, Power distribution
- VHDL/Verilog Simulation
  - Schematic Editor
  - Embedded System (Area, Power, Cost)

Number System

© Osmonov M.S.
KRSU

Outline

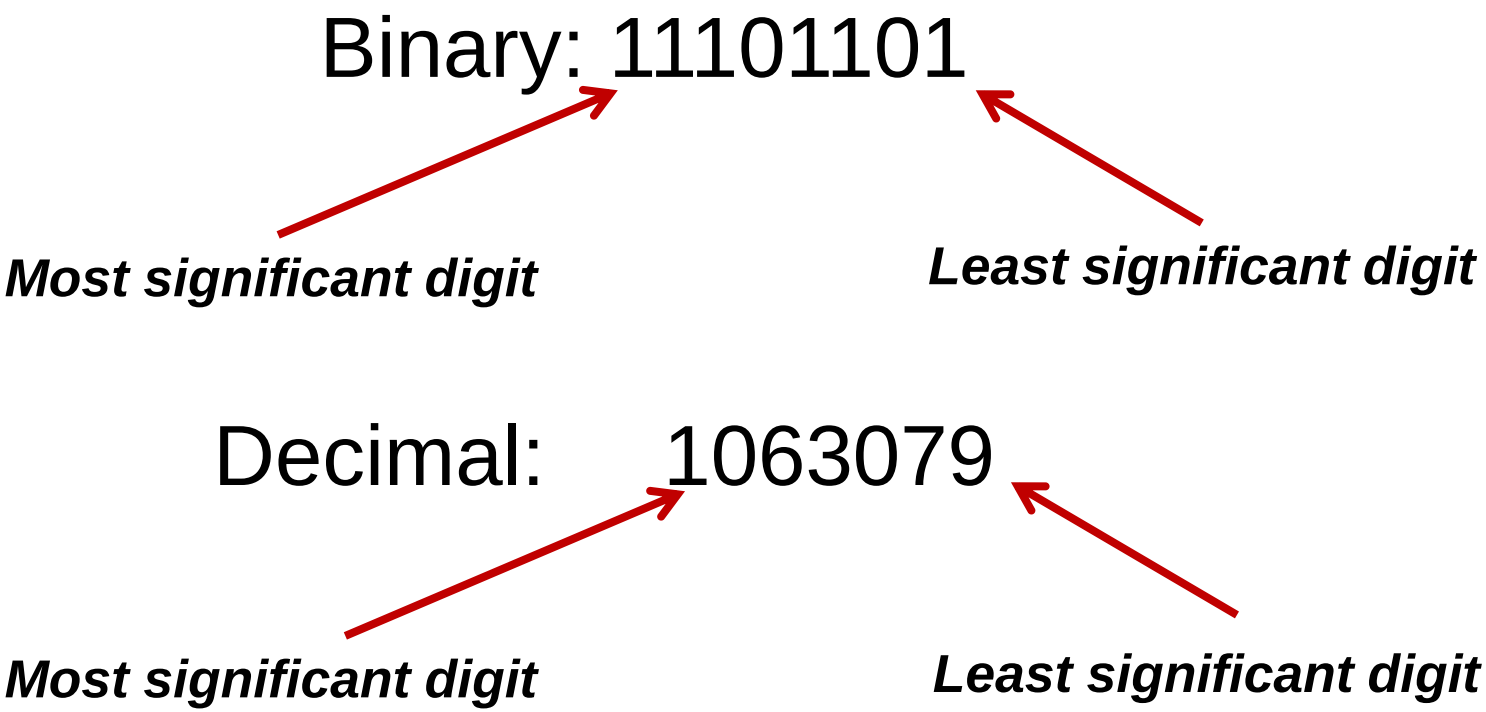
- Number System
  - Decimal, Binary, Octal, Hex
- Conversion (one to another)
  - Decimal to Binary, Octal, Hex & Vice Versa
  - Binary to HEX & vice versa
- Other representation
  - Signed, Unsigned, Complement
- Operation
  - Add, Sub, Mul, Div, Mod
- How to handle real number efficiently ?
  - Float, Double

What Digit? => Number System

- Famous Number System: Dec, Rom, Bin
- Decimal System: 0 -9
  - May evolves: because human have 10 finger
- Roman System
  - May evolves to make easy to look and feel
  - Pre/Post Concept: (IV, V & VI) is (5-1, 5 & 5+1)
- Binary System, Others (Oct, Hex)
  - One can cut an apple in to two

© Osmonov M.S.
KRSU

Significant Digits



Decimal (base 10)

- Uses positional representation
- Each digit corresponds to a power of 10 based on its position in the number
- The powers of 10 increment from 0, 1, 2, etc. as you move right to left

$$1{,}479 = 1 * 10^3 + 4 * 10^2 + 7 * 10^1 + 9 * 10^0$$

Binary (base 2)

- Two digits: 0, 1
- To make the binary numbers more readable, the digits are often put in groups of 4

$$1010 = 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0$$
$$= 8 + 2$$
$$= 10$$

$$1100\ 1001 = 1 * 2^7 + 1 * 2^6 + 1 * 2^3 + 1 * 2^0$$
$$= 128 + 64 + 8 + 1$$
$$= 201$$

How to Encode Numbers: Binary Numbers

- Working with binary numbers
  - In base ten, helps to know powers of 10
    - one, ten, hundred, thousand, ten thousand, ...
  - In base two, helps to know powers of 2
    - one, two, four, eight, sixteen, thirty two, sixty four, one hundred twenty eight
    - Count up by powers of two

2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
512	256	128	64	32	16	8	4	2	1

© Osmonov M.S.
KRSU

Octal (base 8)

- Shorter & easier to read than binary
- 8 digits: 0, 1, 2, 3, 4, 5, 6, 7,
- Octal numbers

$$136_8 = 1 * 8^2 + 3 * 8^1 + 6 * 8^0$$
$$= 1 * 64 + 3 * 8 + 6 * 1$$
$$= 94_{10}$$

Hexadecimal (base 16)

- Shorter & easier to read than binary
- 16 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- “0x” often precedes hexadecimal numbers

$$0x123 = 1 * 16^2 + 2 * 16^1 + 3 * 16^0$$
$$= 1 * 256 + 2 * 16 + 3 * 1$$
$$= 256 + 32 + 3$$
$$= 291_{10}$$

© Osmonov M.S.
KRSU

Counting

Decimal	Binary	Octal	Hexadecimal
0	00000	0	0
1	00001	1	1
2	00010	2	2
3	00011	3	3
4	00100	4	4
5	00101	5	5
6	00110	6	6
7	00111	7	7
8	01000	10	8
9	01001	11	9
10	01010	12	A
11	01011	13	B
12	01100	14	C
13	01101	15	D
14	01110	16	E
15	01111	17	F
16	10000	20	10

Fractional Number

- Point: Decimal Point, Binary Point, Hexadecimal point
- Decimal

  $247.75 = 2x10^{2}+4x10^{1}+7x10^{0}+7x10^{-1}+5x10^{-2}$

- Binary

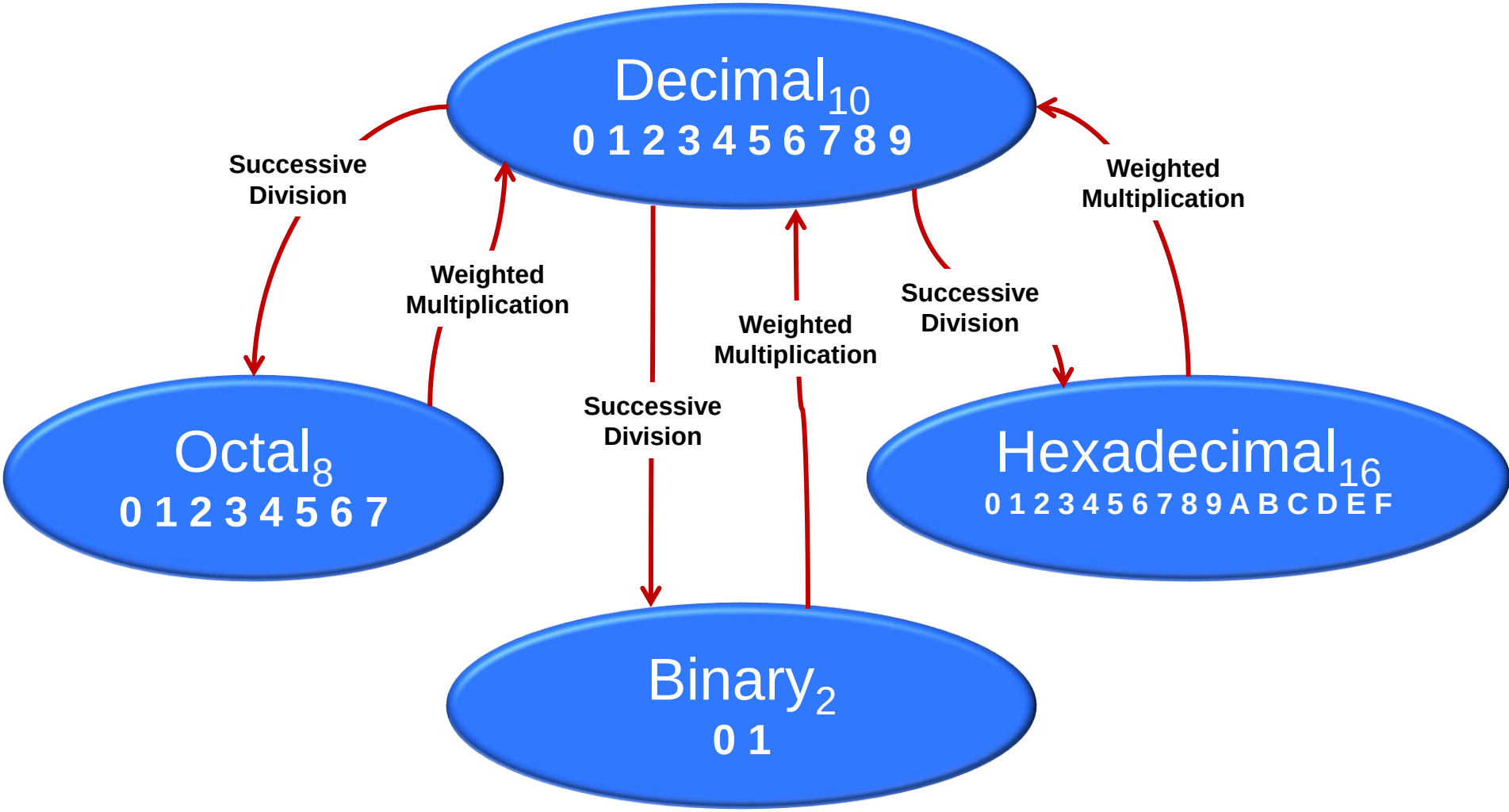
  $10.101= 1x2^{1}+0x2^{0}+1x2^{-1}+0x2^{-2}+1x2^{-3}$

- Hexadecimal

  $6A.7D=6x16^{1}+10x16^{0}+7x16^{-1}+Dx16^{-2}$

© Osmonov M.S.
KRSU

Converting To and From Decimal



Decimal ↔ Binary



- a) Divide the decimal number by **2**; the remainder is the LSB of the **binary** number.
- b) If the quotation is zero, the conversion is complete. Otherwise repeat step (a) using the quotation as the decimal number. The new remainder is the next most significant bit of the **binary** number.



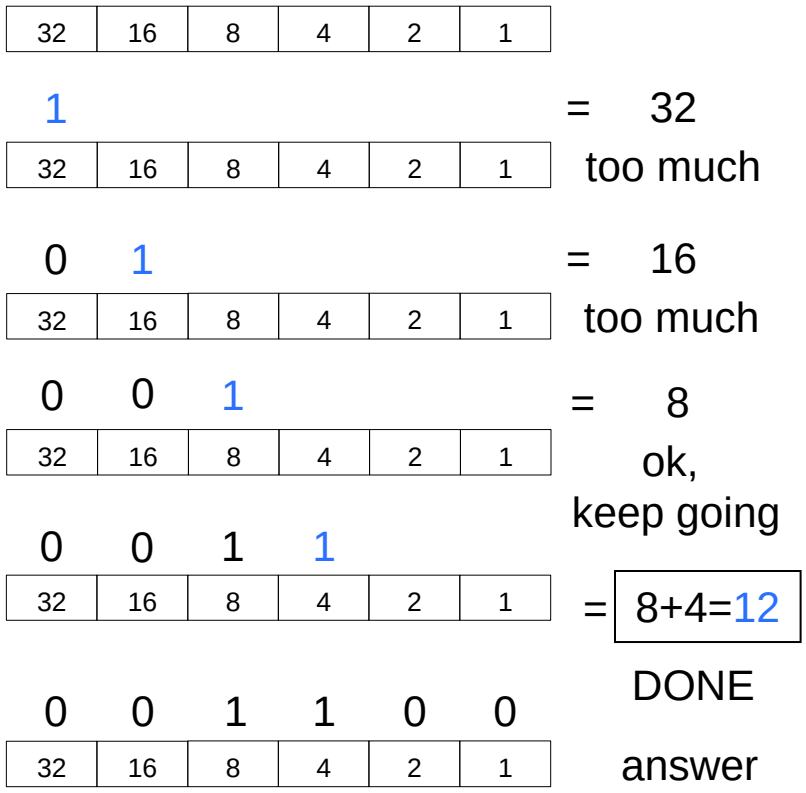
- a) Multiply each bit of the **binary** number by its corresponding bit-weighting factor (i.e., Bit-0→$\mathbf{2}^0$=1; Bit-1→$\mathbf{2}^1$=2; Bit-2→$\mathbf{2}^2$=4; etc.).
- b) Sum up all of the products in step (a) to get the decimal number.

© Osmonov M.S.
KRSU

Decimal to Binary : Subtraction Method

- Goal
  - Good for human
  - Get the binary weights to add up to the decimal quantity
    - Work from left to right
    - (Right to left – may fill in 1s that shouldn't have been there – try it).

- Desired decimal number: **12**



Decimal to Binary : Division Method

- Good for computer: Divide decimal number by 2 and insert remainder into new binary number.
  - Continue dividing quotient by 2 until the quotient is 0.
- Example: Convert decimal number 12 to binary

12 div 2 = (Quotient=6 , Remainder=0) LSB

6 div 2 = (Quo=3, Rem=0)

3 div 2 = (Quo=1,Rem=1)

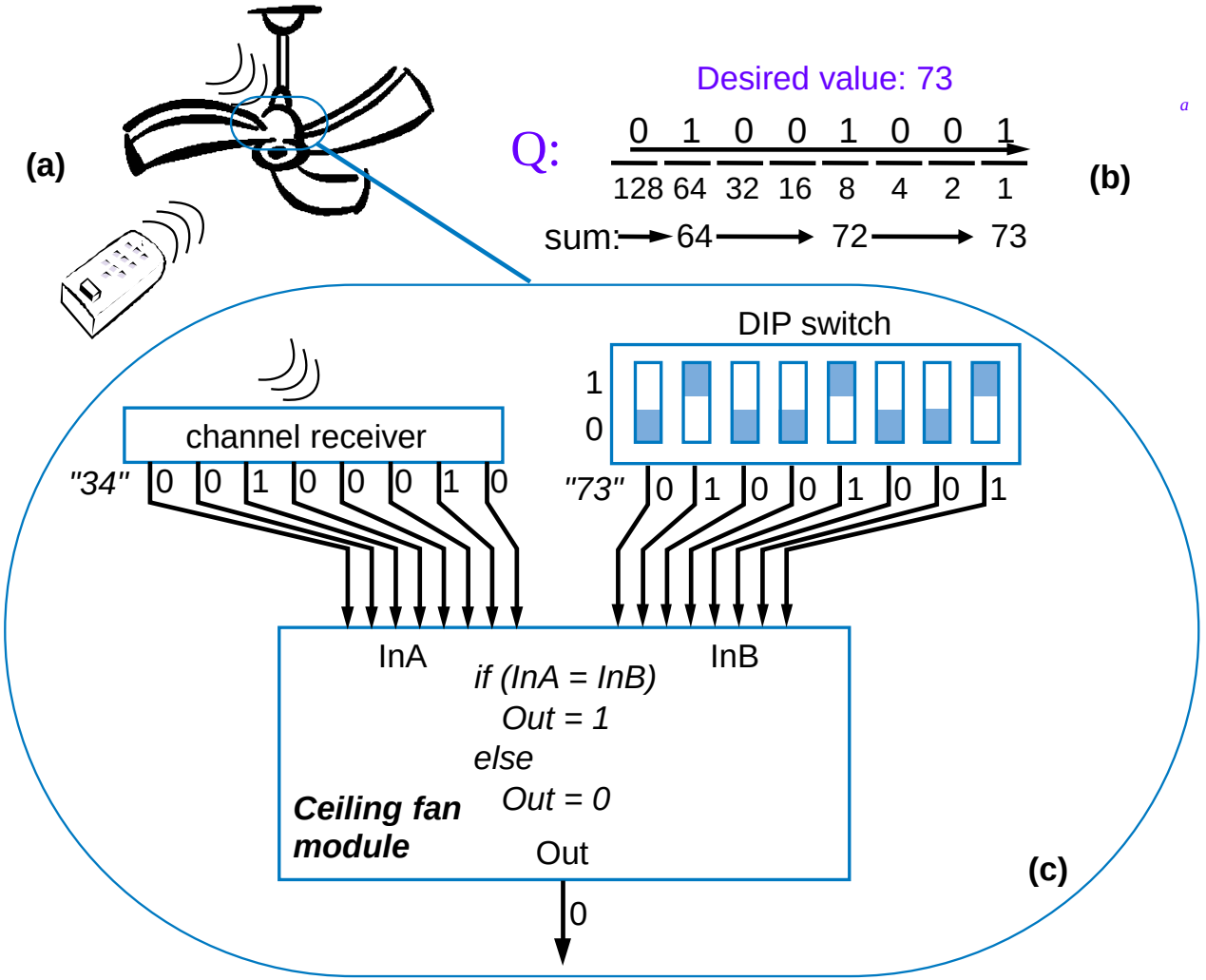
1 div 2 = (Quo=0, Rem=1) MSB

$12_{10} = 1100_{2}$

© Osmonov M.S.
KRSU

Example: DIP-Switch Controlled Channel

- Ceiling fan receiver should be set in factory to respond to channel “73”
- Convert 73 to binary, set DIP switch accordingly



© Osmonov M.S.
KRSU

Conversion Process Decimal ↔ $Base_N$

$Base_{10}$
DECIMAL



$Base_N$
ANY BASE

- a) Divide the decimal number by **N**; the remainder is the LSB of the **ANY BASE** Number .
- b) If the quotient is zero, the conversion is complete. Otherwise repeat step (a) using the quotient as the decimal number. The new remainder is the next most significant bit of the **ANY BASE** number.

$Base_N$
ANY BASE



$Base_{10}$
DECIMAL

- a) Multiply each bit of the **ANY BASE** number by its corresponding bit-weighting factor (i.e., Bit-0→$\mathbf{N}^0$; Bit-1→$\mathbf{N}^1$; Bit-2→$\mathbf{N}^2$; etc.).
- b) Sum up all of the products in step (a) to get the decimal number.

© Osmonov M.S.
KRSU

Decimal ↔ Octal Conversion

- The Process: Successive Division
  - Divide number by **8**; R is the LSB of the **octal** number
  - While Q is 0
    - Using the Q as the decimal number.
    - New remainder is MSB of the **octal** number.

$$
\begin{array}{r|l}
8 & 94 \\
\hline
 & 11 \quad r = 6 \leftarrow \text{LSB}
\end{array}
$$

$$
\begin{array}{r|l}
8 & 11 \\
\hline
 & 1 \quad r = 3
\end{array}
$$

$$
\begin{array}{r|l}
8 & 1 \\
\hline
 & 0 \quad r = 1 \leftarrow \text{MSB}
\end{array}
$$

$$94_{10} = 136_{8}$$

© Osmonov M.S.
KRSU

Decimal ↔ Hexadecimal Conversion

- The Process: Successive Division
  - Divide number by **16**; R is the LSB of the **hex** number
- While Q is 0
  - Using the Q as the decimal number.
  - New remainder is MSB of the **hex** number.

$$16\overline{)\,94}\quad r = E \leftarrow \text{LSB}$$

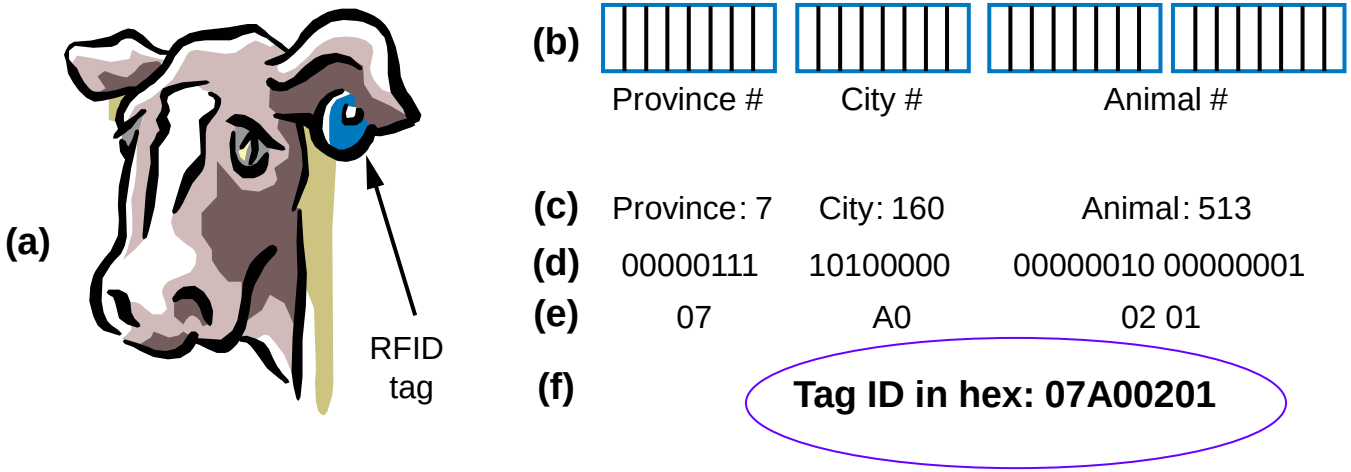
$$16\overline{)\,5}\quad r = 5 \leftarrow \text{LSB}$$

$$94_{10} = 5E_{16}$$

© Osmonov M.S.
KRSU

Hex Example: RFID Tag

- Batteryless tag powered by radio field
  - Transmits unique identification number
  - Example: 32 bit id
    - 8-bit province number, 8-bit country number, 16-bit animal number
    - Tag contents are in binary
    - But programmers use hex when writing/reading



© Osmonov M.S.
KRSU

Example: Hex → Octal

- *Example*:
  - Convert the hexadecimal number $5A_H$ into its octal equivalent.
- *Solution*:
  - First convert the hexadecimal number into its decimal equivalent then convert the decimal number into its octal equivalent.

5	A
16^1	16^0
16	1

80 + 10 = 90_{10}

$5A_H = 132_8$

11		
8) 90	r = 2	← LSB
1		
8) 11	r = 3	
0		
8) 1	r = 1	← MSB

© Osmonov M.S.
KRSU

Example: Octal → Binary

- *Example*:
  - Convert the octal number 132_8 into its binary equivalent.
- *Solution*:
  - First convert the octal number into its decimal equivalent, then convert the number into its binary equivalent decimal.

1	3	2	
8^2	8^1	8^0	
64	8	1	
64	24	2	= 90_{10}

$$132_8 = 1011010_2$$

	45		
2)	90	r = 0	← LSB
	22		
2)	45	r = 1	
	11		
2)	22	r = 0	
	5		
2)	11	r = 1	
	2		
2)	5	r = 1	
	1		
2)	2	r = 0	
	0		
2)	1	r = 1	← MSB

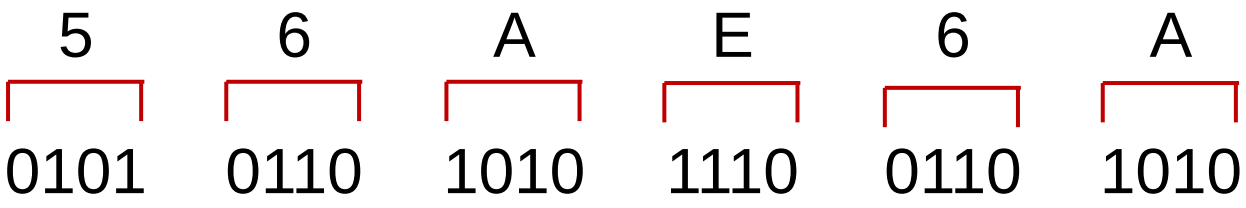
© Osmonov M.S.
KRSU

Binary ↔ Octal ↔ Hex Shortcut

- Relation
  - Binary, octal, and hex number systems
  - All powers of two
- Exploit (This Relation)
  - Make conversion easier.

Substitution Code

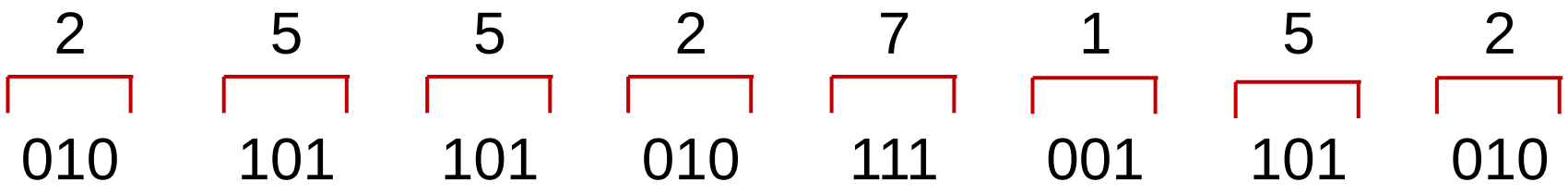
- Convert $01010110101011100110101010_2$ to hex using the 4-bit substitution code :



$56AE6A_{16}$

Substitution Code

- Substitution code can also be used to convert binary to octal by using 3-bit groupings:



25527152_8

Other Representation

- Signed & Unsigned Number
- Signed number last bit (one MSB) is signed bit
- Assume: 8 bit number
  - Unsigned 12 : 0000 1100
  - Signed +12 : 0000 1100
  - Signed -12 : 1000 1100
- Complement number
  - Unsigned binary 12 = 00001100
  - 1's Complement of 12 = 1111 0011

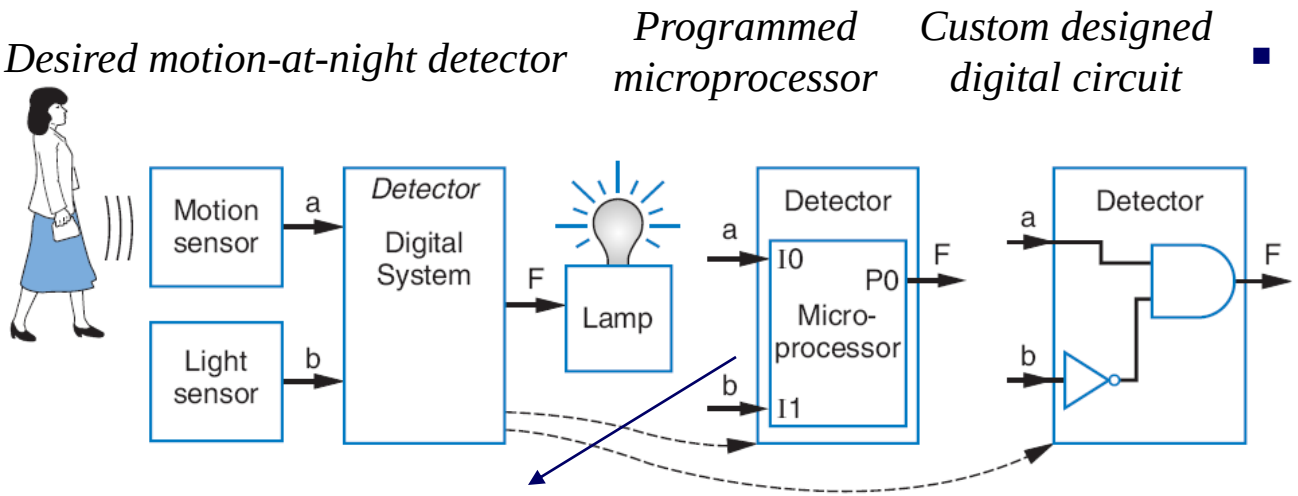
© Osmonov M.S.
KRSU

Bytes, Kilobytes, Megabytes, and More

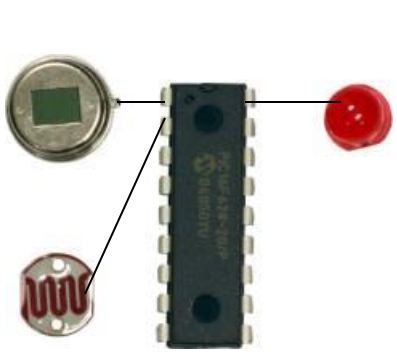
- Byte: 8 bits 
- Common metric prefixes:
  - kilo (thousand, or 10^3), mega (million, or 10^6), giga (billion, or 10^9), and tera (trillion, or 10^{12}), e.g., kilobyte, or KByte
- BUT, metric prefixes also commonly used inaccurately
  - 2^{16} = 65536 commonly written as “64 Kbyte”
  - Typical when describing memory sizes
- Also watch out for “KB” for kilobyte vs. “Kb” for kilobit

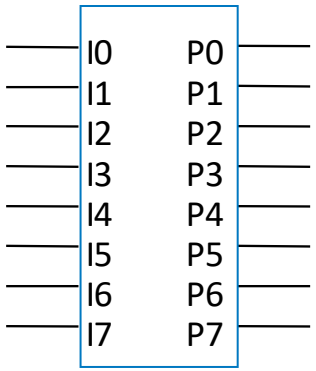
© Osmonov M.S.
KRSU

Implementing Digital Systems: Programming Microprocessors Vs. Designing Digital Circuits

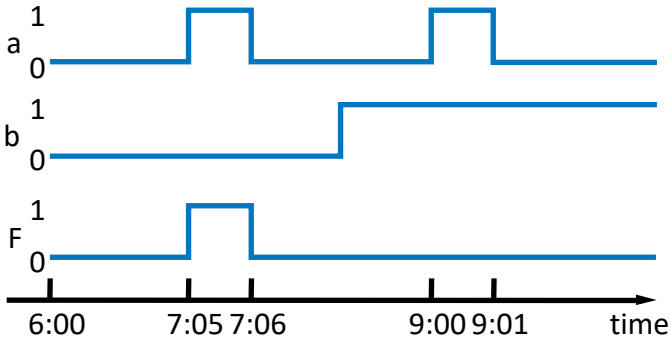


- Microprocessors a common choice to implement a digital system
  - Easy to program
  - Cheap (as low as $1)
  - Readily available






```
void main()
{
  while (1) {
    P0 = I0 && !I1;
    // F = a and !b,
  }
}
```



Digital Design: When Microprocessors Aren't Good Enough

- With microprocessors so easy, cheap, and available, why design a digital circuit?
  - Microprocessor may be too slow
  - Or too big, power hungry, or costly



- Wing controller computation task:
  - 50 ms on microprocessor
  - 5 ms as custom digital circuit
- If must execute 100 times per second:

  100 * 50 ms = 5000 ms = 5 seconds

  100 * 5 ms = 500 ms = 0.5 seconds

Microprocessor too slow, circuit OK.

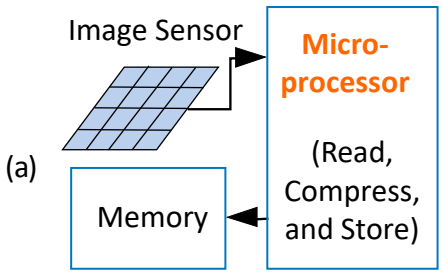
© Osmonov M.S.
KRSU

Digital Design: When Microprocessors Aren't Good Enough

- Commonly, designers partition a system among a microprocessor and custom digital circuits

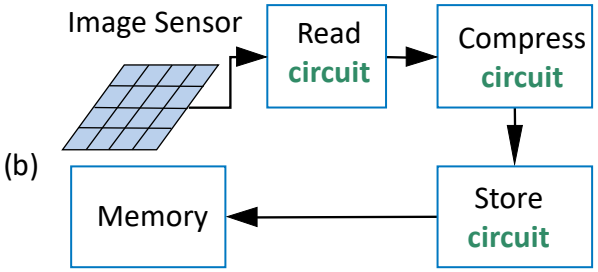
Sample digital camera task execution times (in seconds) on a microprocessor versus a digital circuit:

Task	Microprocessor	Custom Digital Circuit
Read	5	0.1
Compress	8	0.5
Store	1	0.8

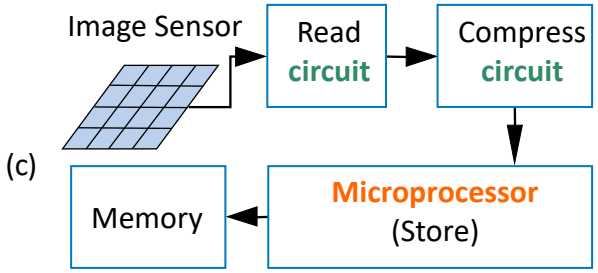


Q: How long for each implementation option?

5+8+1
=14 sec



.1+.5+.8
=1.4 sec



.1+.5+1
=1.6 sec

Good compromise

Summary

- Digital systems surround us
  - Inside computers
  - Inside many other electronic devices (embedded systems)
- Digital systems use 0s and 1s
  - Encoding analog signals to digital can provide many benefits
    - e.g., audio—higher-quality storage/transmission, compression, etc.
  - Encoding integers as 0s and 1s: Binary numbers
- Microprocessors (themselves digital) can implement many digital systems easily and inexpensively
  - But often not good enough—need custom digital circuits

© Osmonov M.S.
KRSU

The lecture is over



Thank you for your attention!!!

© Osmonov M.S.
KRSU